

Programming Assignment 5—Optimization Passes

Marc Lin

1 Constant Branch Folding

For constant branch folding, I simply followed pseudo code to implement function, substitute a conditional branch which is conditional on a constant value, with a unconditional branch.

For the condition value I use *isOne* to check, then use *BranchInst::Create* to create unconditional branch (with only if-true, insertion point parameters).

```
Value* v = br->getCondition();
ConstantInt *cond = dyn_cast<ConstantInt>(v);
BasicBlock *parent = br->getParent();
BasicBlock *leftSucc = br->getSuccessor(0); // 0 for left (cond value = 1, true)
BasicBlock *rightSucc = br->getSuccessor(1); // 1 for right (cond value = 0, true)

if (cond->isOne()) { // if br's condition is true, go to left successor, notify right.
    BranchInst::Create(leftSucc, parent);
    rightSucc->removePredecessor(parent);
}
else { // if br's condition is false, go to right successor, notify left.
    BranchInst::Create(rightSucc, parent);
    leftSucc->removePredecessor(parent);
}
```

2 Dead Block Removal

By implementing DFS to entry block, we can figure out which BB is unreachable.

LLVM already provided *df_ext_begin* and *df_ext_end*, iterate it so we can get DFS-order BB, while adding BB to *visitedSet*.

```
// use df_ext_begin, df_ext_end instead
// doing DFS and add to visitedSet automatically
for (auto it = df_ext_begin(&F.front(), visitedSet) ; it != df_ext_end(&F.front(), visitedSet)
    // do nothing
    // because DepthFirstIterator automatically filled it to the set.
}
```

After getting *visitedSet*, we can also get *unreachableSet*, and notify their successors that they are no longer their predecessors.

3 Speculative Loop Invariant Code Motion (SpecLICM)

Since there are many member functions, I will only mention some functions that have most steps to implement or have some extra awareness that is not mentioned in homework instructions to pass the test.

hoistRegion

This part can be completed by following pseudo code. Here I used *getParent* to check if instruction is moved out of BB.

```
if (currentInstr->getParent() != bb) {  
    continue; // if moved out of bb, just skip  
}
```

Also, when we have nested loop, it is possible to revisit the inner loop again, making two aliases. Therefore, I add another if-statement inside *inCurrentLoop*, make sure we only handle current level of loop, don't go into subloop.

```
if (inCurrentLoop(bb)){  
    // only handle this loop level, not inner loop  
    // avoid revisit of nested loop  
    if (loopInfo->getLoopFor(bb) == currLoop) {
```

canHoistInst

Return true if one of the following is true:

- If it is unordered *LoadInst*, **either** points to a constant memory location **or** must not potentially conflict with any store within the same loop.
- It is one of the following instruction classes: *BinaryOperator*, *CastInst*, *SelectInst*, *GetElementPtrInst*, and *CmpInst*

```
Value* op = LI->getPointerOperand();  
uint64_t loadSize = dl->getTypeStoreSize(LI->getType());  
// not point to constant memory  
// exists conflict  
if (!laa->pointsToConstantMemory(op) &&  
    aliasSetTracker->getAliasSetForPointer(op, loadSize, nullptr).isMod()) {  
    return false;  
}
```

For *getAliasSetForPointer*, I need to get load size, so use *getTypeStoreSize* in *DataLayout*.

specHoistInst

This is the most complex one. Some of the important parts I have done to pass the test are shown below:

For Step 1, we need to use *header->replaceAllUsesWith(newHeader)*. However,

if exiting BB has a PhiNode that records *header*, it will also be changed, but the branch from exiting BB back to header, is still in the original *header*. Therefore we have to recover Phinode in exiting BB, by using TerminatorInst in *header* and get all successors.

```
header->replaceAllUsesWith(newHeader);

TerminatorInst *TI = header->getTerminator();
for (unsigned i = 0; i < TI->getNumSuccessors(); ++i) {
    BasicBlock *succ = TI->getSuccessor(i);
    for (BasicBlock::iterator II = succ->begin(); isa<PHINode>(II); ++II) {
        PHINode *PN = cast<PHINode>(II);
        // if source of this PhiNode is mischanged as newHeader, change it back to header
        for (unsigned j = 0; j < PN->getNumIncomingValues(); ++j) {
            if (PN->getIncomingBlock(j) == newHeader) {
                PN->setIncomingBlock(j, header);
            }
        }
    }
}
```

In Step 4, *ICmpInst* need to be declared as “neg.alias”.

```
// (4) Use StoreInst, LoadInst to operate memory in stack
Value *constZero = Constant::getNullValue(Type::getInt1Ty(ctx));
new StoreInst(constZero, alias, preheader->getTerminator());
new StoreInst(constZero, alias, fixupBB->getTerminator());

LoadInst *aliasVal = new LoadInst(alias, "alias.ld", newHeader);
// insertpoint, operator, op1, op2
ICmpInst *cmp = new ICmpInst(*newHeader, CmpInst::ICMP_EQ, aliasVal, constZero, "neg.alias");
BranchInst::Create(header, fixupBB, cmp, newHeader); // if-true, if-false, cmp, branchpoint
```

In Step 5, since we want to deal with successors from same predecessor, we handle them from left to right. Therefore, it is actually queue-based (FIFO) worklist instead of stack-based worklist.

```
// (5) insert checking code after each store that may conflict with the input load
AliasSet &AS = aliasSetTracker->getAliasSetForPointer(op, loadSize, nullptr);
std::vector<Instruction*> targetSI;
std::queue<Instruction*> worklist;
```

```
// queue-based recursive traversal (recursive-DFS-like)
while (!worklist.empty()) {
    Instruction *curr = worklist.front();
    worklist.pop();
```

In addition, *ICmpInst* should be before the store, the action that store cmp to alias should be after *StoreInst*.

fillFixupBlocks

When iterating user list, we also want to update user's dependency from *inst* to the new *LoadInst li*. In case to avoid altering a list during traversing, first make a copy of user list, then we alter the real one.

```
AllocaInst *ai = nullptr;

SmallVector<User*, 8> userList(inst->users());
for (User *U : userList) {
    Instruction *user = dyn_cast<Instruction>(U);
```

computeFrequentPath

At the start, we need to get header of loop, but the member variable *header* hasn't been initialized yet (it is called earlier than initialization in *runOnLoop*), so just use *L->getHeader()*. I also use *getNumSuccessors* to determine whether a loop header has only one successor or has multiple.

```
frequentPath.clear();
BasicBlock *loopHeader = L->getHeader();
frequentPath.insert(loopHeader);
BasicBlock *cur = loopHeader;

while (true) {
    BasicBlock *next = nullptr;
    TerminatorInst *ter = cur->getTerminator();
    unsigned numSuccs = ter->getNumSuccessors();
```

To stop traversal, we have 3 conditions and only required one of them is true. If next BB is on hot path, means it forms a cycle.

```
// 1. Reaches the backedge: next == header
// 2. Leaves the current loop: !currLoop->contains(next)
// 3. Creates a cycle: already in frequentPath
bool isCycle = false;
for (auto *BB : frequentPath) {
    if (BB == next) {
        isCycle = true;
        break;
    }
}
if (next == loopHeader || !currLoop->contains(next) || isCycle) {
    break;
}
```